

# EVENTFORCE MANAGEMENT SYSTEM

*Salesforce Implementation Project Documentation*

---

**Team Leader:** Yogeswari S

**Team Members:** Vaishnavi G, Vijaysree A, Pabhisri V, Sruthi M

**College:** Arasu Engineering College

**Registered Mail ID:** shanyoges2005@gmail.com

**Team ID:** SWTID-2026-3039

**Platform:** Salesforce Developer Edition Org (Lightning Experience)

**Submission Date:** 3rd October 2026

## Contents

1. Project Overview
  2. Objectives
  3. Problem Statement and Requirements
  4. Project Planning
  5. Implementation (Setup, Data Model, Rules, Automation, Apex, Interface, Security)
  6. Testing
  7. Challenges Faced and Fixes
  8. Limitations and Future Scope
  9. Conclusion
- Appendix A: Apex Source Code

## 1. Project Overview

EventForce Management System is a Salesforce CRM application designed for an event planning business. In most small event companies the details of clients, bookings, venues and vendors sit in different places such as notebooks, spreadsheets and chat messages, which leads to missed follow-ups and venue clashes. EventForce moves all of this into a single Salesforce org so that every team member works from one shared and current set of records.

The application is built from six custom objects (Event, Client, Vendor, Venue, Feedback and Event Vendor) joined by lookup and master-detail relationships. On top of this data model we added a validation rule that checks client email addresses, a formula field for the event budget, an approval process for cancellation requests, and a record-triggered Flow that emails the client three days before an event. Apex code keeps each venue's availability status in step with its events, refuses a second booking for the same venue on the same date, and a scheduled batch job closes finished events automatically. A report and an operations dashboard summarise upcoming events for management, and everything is grouped inside a Lightning App named Event planner.

All configuration and code was created and tested by our team inside a free Salesforce Developer Edition org, working in Lightning Experience.

## 2. Objectives

- Hold client, event, venue, vendor and feedback data in one structured system.
- Cut down manual effort when booking events and reserving venues.
- Make sure the same venue cannot be booked twice for the same date.
- Remind clients automatically a few days before their event.
- Send cancellation requests through a controlled approval process with notifications.
- Close completed events automatically without anyone having to remember.
- Give management a quick overview through a dashboard.
- Limit what each kind of user can see and do.

## 3. Problem Statement and Requirements

### 3.1 Problem Statement

An event coordinator deals with many clients, venues and vendors at once. Without a central system the coordinator has to check several lists to find out whether a venue is free, has to remember to remind each client by hand, and has no formal route for approving a cancellation. Mistakes such as a double-booked venue or a forgotten reminder damage the company's reputation and leave clients unsure whether their event is really confirmed.

### 3.2 Users and Their Needs

| User               | What they need from the system                                                                      |
|--------------------|-----------------------------------------------------------------------------------------------------|
| Client             | To book an event, receive a reminder before it, and give feedback afterwards.                       |
| Event Coordinator  | To create and manage events, pick venues and clients, and be told when a cancellation is requested. |
| Manager (approver) | To review cancellation requests and approve or reject them.                                         |
| Administrator      | To configure the org, manage security and monitor the scheduled jobs.                               |

### 3.3 Functional Requirements

| ID    | Requirement                                                                         | Delivered by                                                           |
|-------|-------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| FR-1  | Store clients, events, venues, vendors and feedback as related records.             | Six custom objects                                                     |
| FR-2  | Reject an invalid client email address.                                             | Validation rule Email_Valid_Address                                    |
| FR-3  | Calculate the event budget automatically from the event type.                       | Formula field Event_Budget__c                                          |
| FR-4  | Block two events at one venue on the same date.                                     | Trigger PreventDoubleBooking                                           |
| FR-5  | Mark a venue Reserved when an event is confirmed and Available when it is canceled. | EventTrigger13 and VenueStatusHelper                                   |
| FR-6  | Email the client three days before the event.                                       | Flow: Client Reminder - 3 Days Before                                  |
| FR-7  | Route a cancellation through approval.                                              | Approval process<br>Event_Cancellation_Process                         |
| FR-8  | Complete past events automatically every day.                                       | BatchCompleteEvents and<br>ScheduleCompleteEvents                      |
| FR-9  | Give management an overview of upcoming events.                                     | Report Upcoming Events by Month and<br>EventForce Operations Dashboard |
| FR-10 | Control access by role, profile and sharing.                                        | Security configuration (Section 5.9)                                   |

### 3.4 Technology Stack

| Layer                   | Technology used                                                                      |
|-------------------------|--------------------------------------------------------------------------------------|
| Platform                | Salesforce Developer Edition, Lightning Experience                                   |
| Data model              | Custom objects, custom fields, lookup and master-detail relationships                |
| Declarative automation  | Validation rule, formula field, approval process, email alert, record-triggered Flow |
| Programmatic            | Apex triggers, Apex helper class, Batch Apex and Schedulable Apex                    |
| Interface and analytics | Lightning App (Event planner), tabs, record forms, report, dashboard                 |

| Layer | Technology used                                                                          |
|-------|------------------------------------------------------------------------------------------|
| Tools | Setup, Object Manager, Flow Builder, Developer Console / Apex Classes and Triggers pages |

### 3.5 How Data Moves Through the System

| Step | What happens                                     | Result                                                             |
|------|--------------------------------------------------|--------------------------------------------------------------------|
| 1    | A client record is saved with an email address.  | Validation rule accepts or rejects the email.                      |
| 2    | An event is saved with a client and a venue.     | PreventDoubleBooking checks other events at that venue.            |
| 3    | The event status becomes Confirmed or Canceled.  | Venue status changes to Reserved or Available.                     |
| 4    | The event date is three days away.               | The Flow's scheduled path sends the reminder email.                |
| 5    | The event status is set to Pending Cancellation. | Approval process starts and the Event Coordinator role is alerted. |
| 6    | Every evening the scheduled job runs.            | BatchCompleteEvents processes events in batches of 200.            |
| 7    | Management opens the report or dashboard.        | Upcoming events are shown month by month.                          |

## 4. Project Planning

The dates below are taken from the creation and modification timestamps shown in our Salesforce org, so they reflect when each piece of work was actually done.

| Task                                                             | Date                | Evidence in the org                                          |
|------------------------------------------------------------------|---------------------|--------------------------------------------------------------|
| Create the Developer Edition org and the six custom objects      | 29 Sep 2026         | Object Manager: all six objects last modified 9/29/2026      |
| Create the Event Budget formula field                            | 29 Sep 2026         | Field created 9/29/2026, 2:27 AM                             |
| Create the client email validation rule                          | 29 Sep 2026         | Rule created 9/29/2026, 3:44 AM                              |
| Build the cancellation approval process                          | 29 Sep 2026         | Created 4:00 AM, modified 9:21 PM                            |
| Write VenueStatusHelper, EventTrigger13 and PreventDoubleBooking | 29 Sep 2026         | Created between 9:25 PM and 9:32 PM                          |
| Write and schedule the nightly batch job                         | 29 Sep 2026         | ScheduleCompleteEvents created 10:11 PM; job queued 10:18 PM |
| Create the EventForce Operations Dashboard                       | 29 Sep 2026         | Dashboard created 10:47 PM                                   |
| Build and activate the 3-day reminder Flow                       | 30 Sep 2026         | Flow last saved 9/30/2026, 10:25 AM                          |
| Observe the scheduled batch runs                                 | 30 Sep - 2 Oct 2026 | Apex Jobs: completed runs on 9/30, 10/1 and 10/2             |
| Testing and documentation                                        | 1 - 3 Oct 2026      | This document                                                |

## 5. Implementation

### 5.1 Developer Org Setup

We signed up for a free Developer Edition org, verified the account by email and logged in to Lightning Experience. Every object, rule, flow and class described in this document was created in this org through the Setup menu.

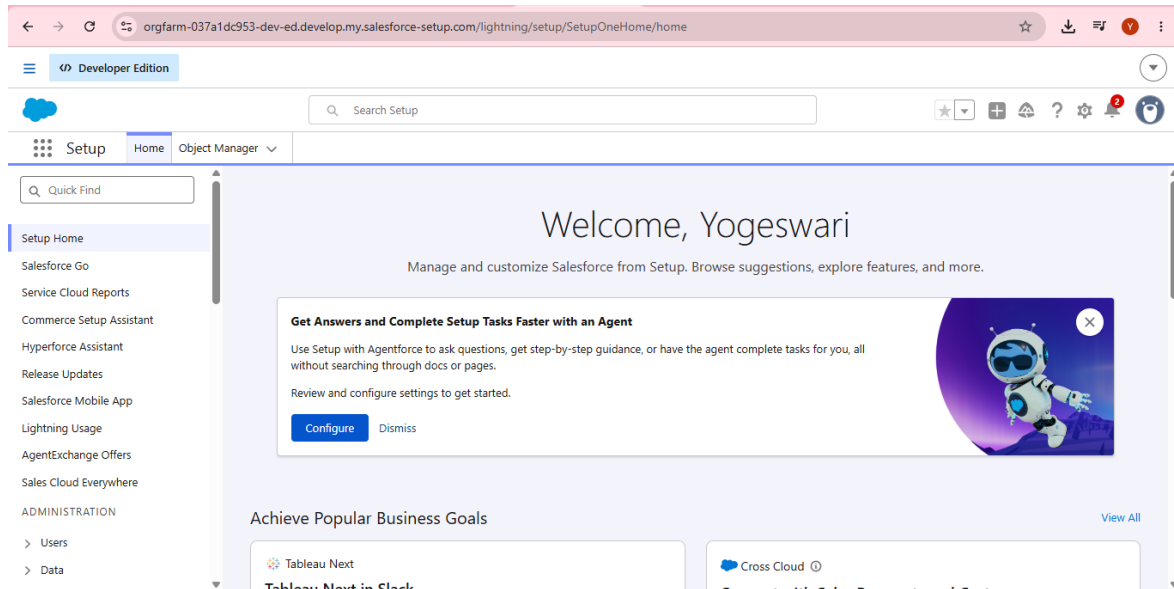


Figure 1: Setup Home page of the Developer Edition org

### 5.2 Data Model

The Object Manager lists the six custom objects that make up EventForce. All of them show as deployed and were created on 29 September 2026.

The screenshot shows the Salesforce Object Manager page. The browser address bar displays 'orgfarm-037a1dc953-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/home'. The page title is 'Object Manager' with a subtitle '54+ Items, Sorted by Last Modified'. A table lists six custom objects: Event Vendor, Feedback, Venue, Vendor, Client, and Event. Each row includes columns for Label, API Name, Type, Description, Last Modified, and Deployed status. All objects are marked as 'Deployed' with a checkmark. A 'Work Type Group Member' object is also visible at the bottom of the list.

| LABEL                  | API NAME            | TYPE            | DESCRIPTION | LAST MODIFIED | DEPLOYED |
|------------------------|---------------------|-----------------|-------------|---------------|----------|
| Event Vendor           | Event_Vendor__c     | Custom Object   |             | 9/29/2026     | ✓        |
| Feedback               | Feedback__c         | Custom Object   |             | 9/29/2026     | ✓        |
| Venue                  | Venue__c            | Custom Object   |             | 9/29/2026     | ✓        |
| Vendor                 | Vendor__c           | Custom Object   |             | 9/29/2026     | ✓        |
| Client                 | Client__c           | Custom Object   |             | 9/29/2026     | ✓        |
| Event                  | Event__c            | Custom Object   |             | 9/29/2026     | ✓        |
| Work Type Group Member | WorkTypeGroupMember | Standard Object |             |               |          |

Figure 2: Object Manager showing the six custom objects

| Object (API name)              | Purpose                                            | Fields we created                                                                                         |
|--------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Event (Event__c)               | One record for every booked event.                 | Event Name, Event Type, Event Date, Event Status, Event Budget (formula), Client (lookup), Venue (lookup) |
| Client (Client__c)             | The customer who books an event.                   | Client Name, Email (unique), Phone, Address, City, Country                                                |
| Vendor (Vendor__c)             | A service provider such as catering or decoration. | Vendor Name, Email, Phone, Service Type                                                                   |
| Venue (Venue__c)               | A place where events can be held.                  | Address, Location (URL), Capacity, Availability Status                                                    |
| Feedback (Feedback__c)         | Comments given by a client about an event.         | Feedback Name (auto number), Comments, Event (lookup), Client (lookup)                                    |
| Event Vendor (Event_Vendor__c) | Junction object joining events and vendors.        | Event (master-detail), Vendor (master-detail)                                                             |

Relationships: an Event looks up to one Client and one Venue. Feedback looks up to both the Event and the Client it belongs to. Because one event can use many vendors and one vendor can serve many events, Event Vendor holds two master-detail relationships (to Event and to Vendor), which forms a many-to-many link.

The screenshot shows the Salesforce Setup interface for the 'Event' object. The 'Fields & Relationships' section is active, displaying a list of 10 items sorted by Field Label. The fields listed are:

| FIELD LABEL  | FIELD NAME      | DATA TYPE          | CONTROLLING FIELD | INDEXED |
|--------------|-----------------|--------------------|-------------------|---------|
| Client       | Client__c       | Lookup(Client)     |                   | ✓       |
| Created By   | CreatedById     | Lookup(User)       |                   |         |
| Event Budget | Event_Budget__c | Formula (Currency) |                   |         |
| Event Date   | Event_Date__c   | Date               |                   |         |
| Event Name   | Name            | Text(80)           |                   | ✓       |
| Event Status | Event_Status__c | Picklist           |                   |         |

Figure 3: Event object - fields and relationships

Developer Edition

Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

**Client**

Details

**Fields & Relationships**  
9 Items, Sorted by Field Label

Quick Find New Deleted Fields Field Dependencies Set History Tracking

| FIELD LABEL | FIELD NAME  | DATA TYPE      | CONTROLLING FIELD | INDEXED |
|-------------|-------------|----------------|-------------------|---------|
| Address     | Address__c  | Text Area(255) |                   |         |
| City        | City__c     | Picklist       |                   |         |
| Client Name | Name        | Text(80)       |                   | ✓       |
| Country     | Country__c  | Picklist       |                   |         |
| Created By  | CreatedById | Lookup(User)   |                   |         |
| Email       | Email__c    | Email          |                   |         |

Page Layouts  
Lightning Record Pages  
Buttons, Links, and Actions  
Compact Layouts  
Field Sets  
Object Limits  
Record Types  
Related Lookup Filters

Figure 4: Client object - fields and relationships

Developer Edition

Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

**Vendor**

Details

**Fields & Relationships**  
8 Items, Sorted by Field Label

Quick Find New Deleted Fields Field Dependencies Set History Tracking

| FIELD LABEL      | FIELD NAME       | DATA TYPE          | CONTROLLING FIELD | INDEXED |
|------------------|------------------|--------------------|-------------------|---------|
| Created By       | CreatedById      | Lookup(User)       |                   |         |
| Email            | Email__c         | Email              |                   |         |
| Last Modified By | LastModifiedById | Lookup(User)       |                   |         |
| Owner            | OwnerId          | Lookup(User,Group) |                   | ✓       |
| Phone            | Phone__c         | Phone              |                   |         |
| Service Type     | Service_Type__c  | Picklist           |                   |         |

Page Layouts  
Lightning Record Pages  
Buttons, Links, and Actions  
Compact Layouts  
Field Sets  
Object Limits  
Record Types

Figure 5: Vendor object - fields and relationships

orgfarm-037a1dc953-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01lgK000007eQqz/FieldsAndRelationships/view

Developer Edition

Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

**Venue**

Details

**Fields & Relationships**  
8 Items, Sorted by Field Label

Quick Find New Deleted Fields Field Dependencies Set History Tracking

| FIELD LABEL         | FIELD NAME             | DATA TYPE      | CONTROLLING FIELD | INDEXED |
|---------------------|------------------------|----------------|-------------------|---------|
| Address             | Address__c             | Text Area(255) |                   |         |
| Availability Status | Availability_Status__c | Picklist       |                   |         |
| Capacity            | Capacity__c            | Number(18, 0)  |                   |         |
| Created By          | CreatedById            | Lookup(User)   |                   |         |
| Last Modified By    | LastModifiedById       | Lookup(User)   |                   |         |
| Location            | Location__c            | URL(255)       |                   |         |

https://orgfarm-037a1dc953-dev-ed.develop.my.salesforce-setup.com/one/one.a...

Figure 6: Venue object - fields and relationships

orgfarm-037a1dc953-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ObjectManager/01lgK000007eQsb/FieldsAndRelationships/view

Developer Edition

Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

**Feedback**

Details

**Fields & Relationships**  
8 Items, Sorted by Field Label

Quick Find New Deleted Fields Field Dependencies Set History Tracking

| FIELD LABEL      | FIELD NAME       | DATA TYPE             | CONTROLLING FIELD | INDEXED |
|------------------|------------------|-----------------------|-------------------|---------|
| Client           | Client__c        | Lookup(Client)        |                   | ✓       |
| Comments         | Comments__c      | Long Text Area(32768) |                   |         |
| Created By       | CreatedById      | Lookup(User)          |                   |         |
| Event            | Event__c         | Lookup(Event)         |                   | ✓       |
| Feedback Name    | Name             | Auto Number           |                   | ✓       |
| Last Modified By | LastModifiedById | Lookup(User)          |                   |         |

https://orgfarm-037a1dc953-dev-ed.develop.my.salesforce-setup.com/one/one.a...

Figure 7: Feedback object - fields and relationships

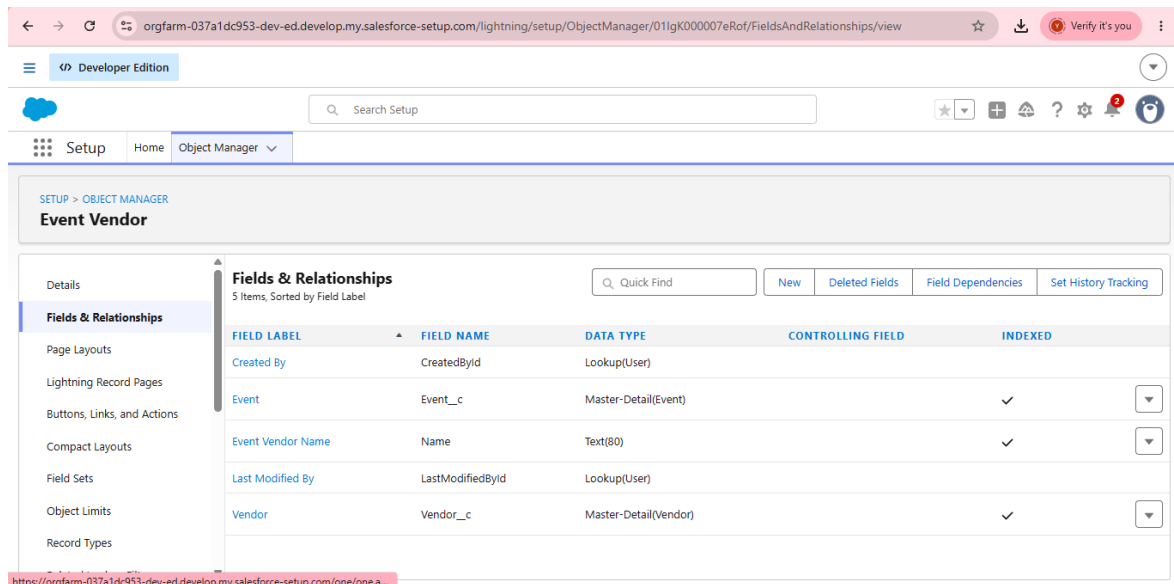


Figure 8: Event Vendor junction object with two master-detail relationships

### 5.3 Validation Rule for Client Email

To keep contact data clean, the Client object has a validation rule named Email\_Valid\_Address. It tests the Email field against a regular expression and blocks the save when the pattern does not match, showing the message “Please Enter Valid Email Address” beside the field.

| Property                | Value                                                               |
|-------------------------|---------------------------------------------------------------------|
| Rule name               | Email_Valid_Address (Active)                                        |
| Object                  | Client                                                              |
| Error condition formula | NOT(Regex(Email__c, "[a-zA-Z0-9._]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}\$")) |
| Error message           | Please Enter Valid Email Address                                    |
| Error location          | Email field                                                         |

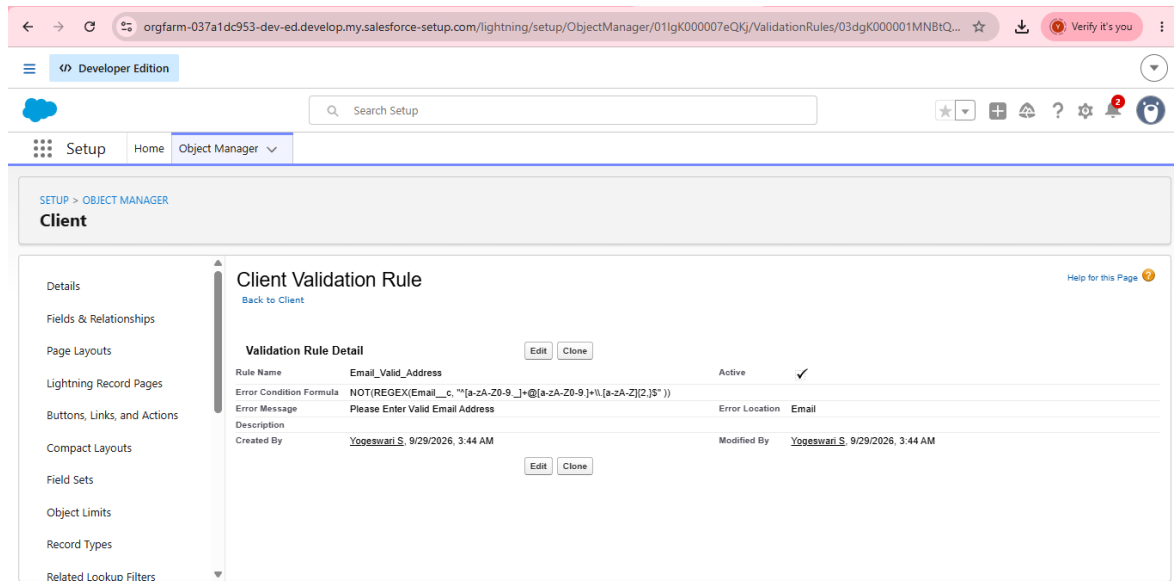


Figure 9: Validation rule Email\_Valid\_Address on the Client object

## 5.4 Event Budget Formula Field

Event Budget on the Event object is a Formula field of type Currency with two decimal places. Because it is a formula, Salesforce works out the value from the Event Type and users cannot overwrite it by hand, so the same type of event always gets the same budget.

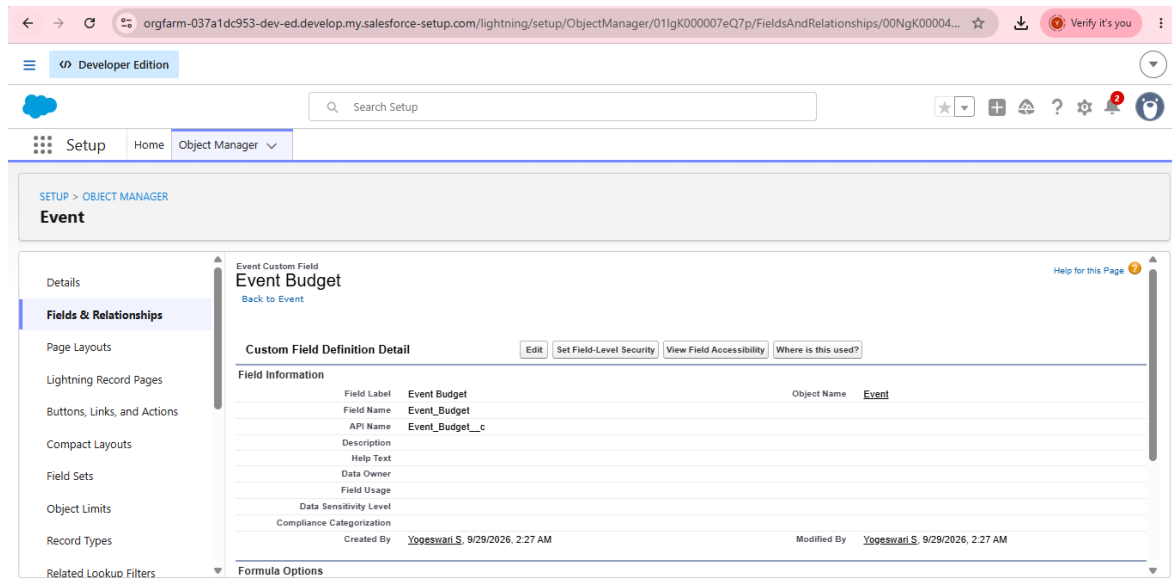


Figure 10: Event Budget formula field (Currency) on the Event object

The formula uses the CASE function on the text value of the Event Type picklist. Any event type that is not listed gets a budget of 0.

| Event Type      | Event Budget |
|-----------------|--------------|
| Wedding         | 50,000       |
| Corporate       | 30,000       |
| Birthday        | 10,000       |
| Anniversary     | 20,000       |
| Festival        | 60,000       |
| Concert         | 40,000       |
| Other           | 15,000       |
| Any other value | 0            |

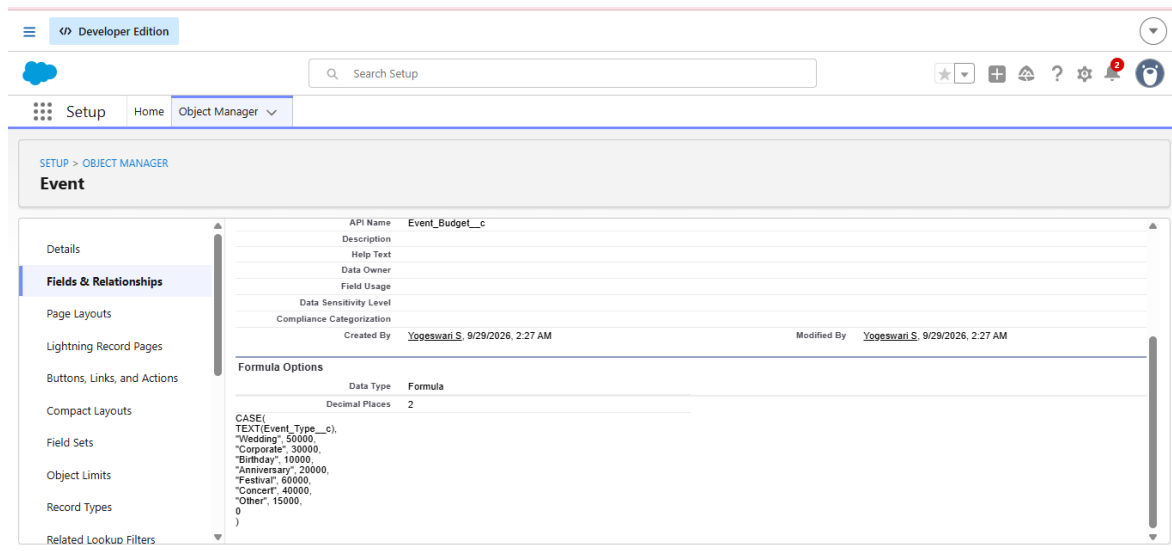


Figure 11: Formula Options showing the CASE expression for Event Budget

## 5.5 Approval Process for Event Cancellation

Cancelling a booking should not be a one-click action, so we created the approval process Event\_Cancellation\_Process on the Event object. A record enters the process when its Event Status is set to Pending Cancellation. The approver is the manager of the person who submitted the record, and while the request is waiting the record can be edited by an administrator only.

| Setting                            | Value                                    |
|------------------------------------|------------------------------------------|
| Process name                       | Event_Cancellation_Process (Active)      |
| Object                             | Event                                    |
| Entry criteria                     | Event Status equals Pending Cancellation |
| Next automated approver            | Manager of Record Submitter              |
| Initial submitters                 | Event Owner                              |
| Record editability while pending   | Administrator only                       |
| Approval assignment email template | Event Cancellation Request Notification  |

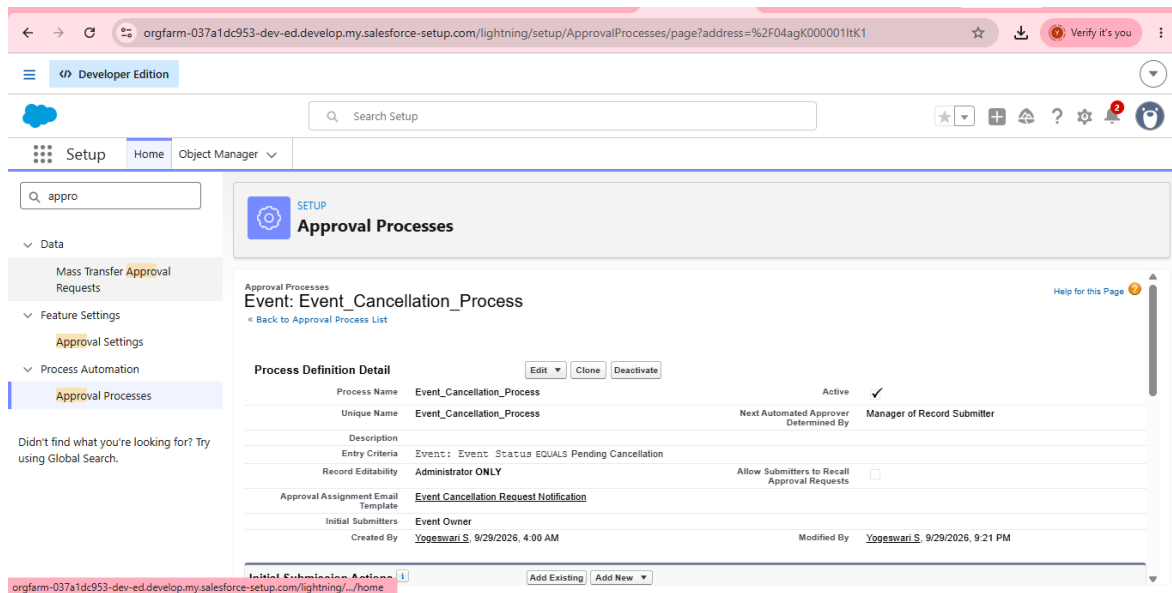


Figure 12: Approval process Event\_Cancellation\_Process

The process also defines what happens at the end. When a request is approved, the final approval actions lock the record, update the Event Status field and send the email alert “Alert - Cancellation Approved”. When a request is rejected, the final rejection actions unlock the record so it can be edited again and send the email alert “Alert - Cancellation Rejected”.

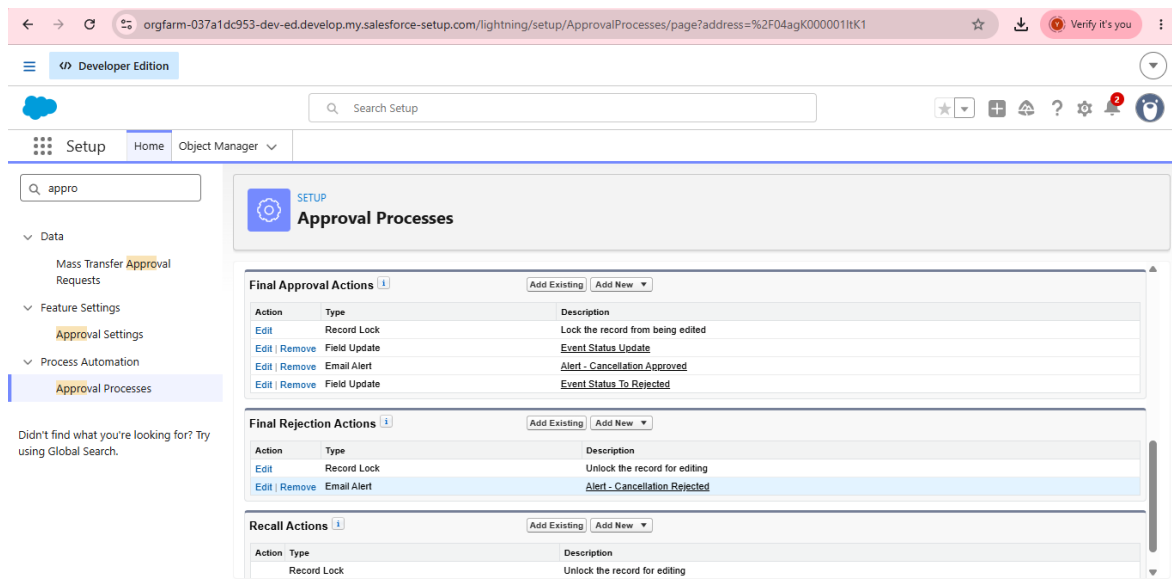


Figure 13: Final Approval Actions and Final Rejection Actions of the process

An email alert named “Alert - Event Cancellation Request” on the Event object uses the Event Cancellation Request template and sends it to everyone in the Event Coordinator role, so the coordinators know as soon as a cancellation is requested.

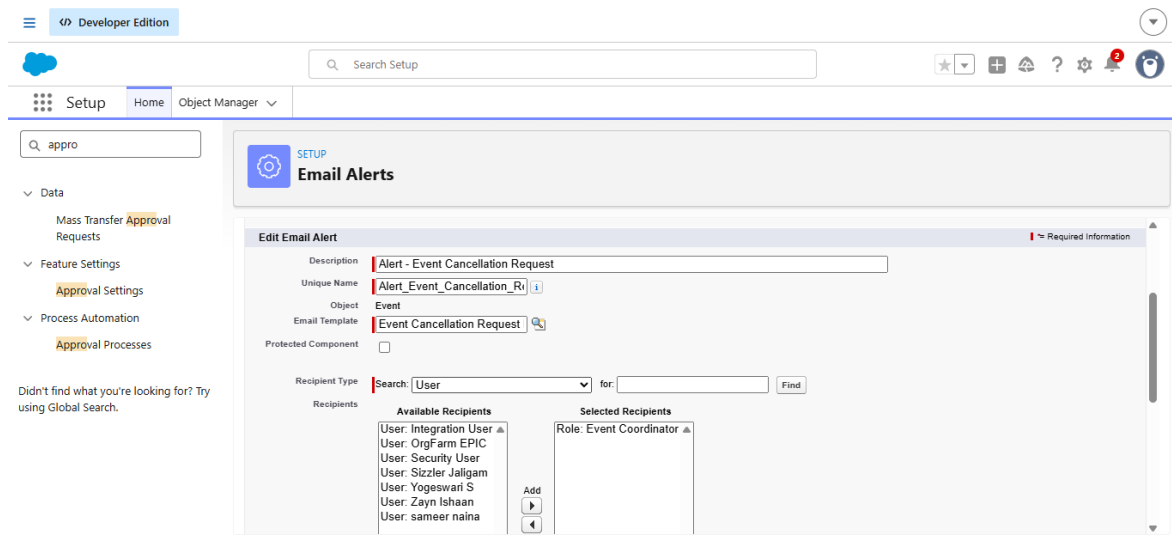


Figure 14: Email alert for event cancellation requests

## 5.6 Record-Triggered Flow: Client Reminder

The Flow “Client Reminder - 3 Days Before” is a record-triggered Flow on the Event object. It starts whenever an event is created or updated and has two paths. The Run Immediately path simply ends. The 3\_Days\_Before scheduled path waits until three days before the event and then runs the action Alert\_Client\_3Day\_Reminder, which sends the reminder email. The Flow is active (version 1, last saved on 30 September 2026).

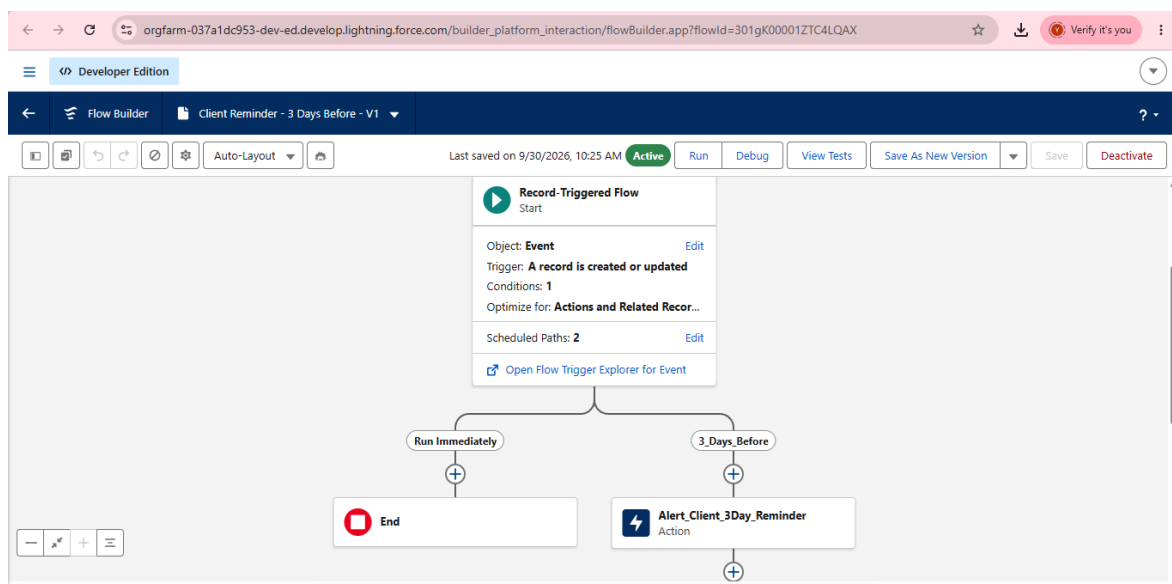


Figure 15: Record-triggered Flow with the 3\_Days\_Before scheduled path

## 5.7 Apex Code

Some rules cannot be expressed with clicks alone, so we wrote Apex for the venue logic and the nightly job.

## Venue status update

EventTrigger13 runs after an Event is inserted or updated and hands the records to the VenueStatusHelper class. For every event that has a venue, the helper sets the venue's Availability Status to Reserved when the event status is Confirmed and back to Available when the status is Canceled.

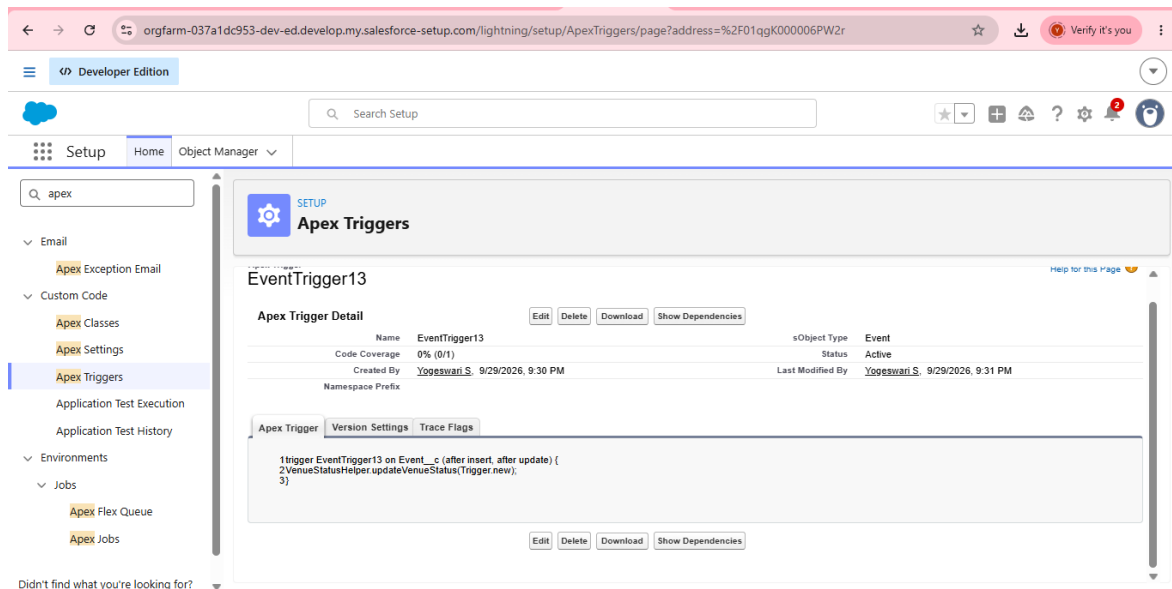


Figure 16: Apex trigger EventTrigger13

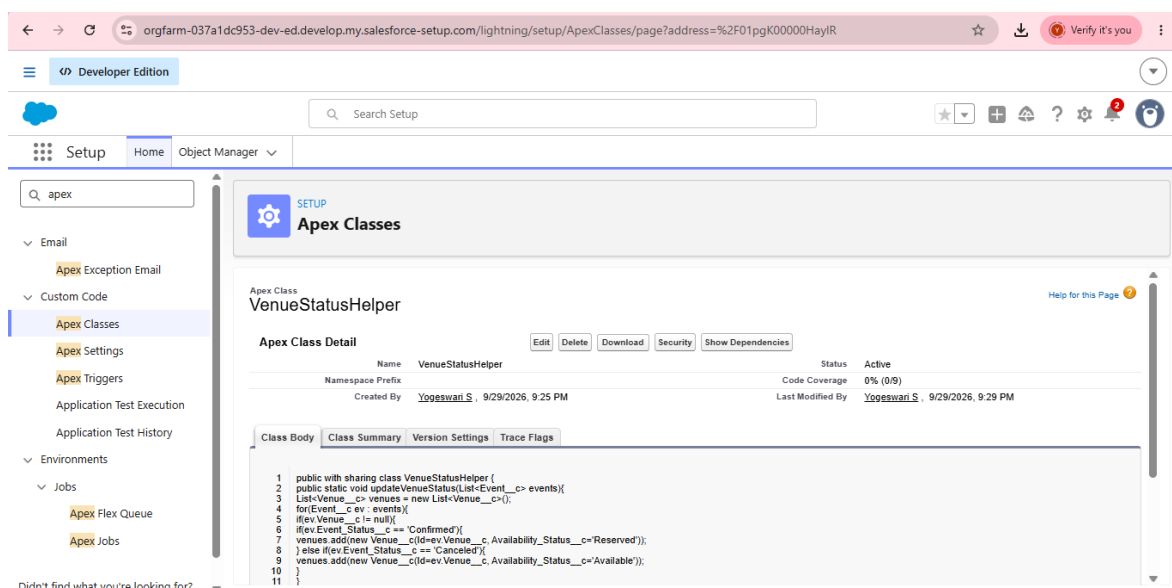


Figure 17: Apex class VenueStatusHelper

## Double-booking prevention

PreventDoubleBooking runs before an Event is inserted or updated. It collects the venue IDs of the incoming events, queries the existing events at those venues and groups them by venue in a map. Then, for each incoming event, it compares the Event Date with the dates of the other events at the same venue (ignoring the record itself). If a date matches, the save is blocked with the message "This Venue is already booked on this date."

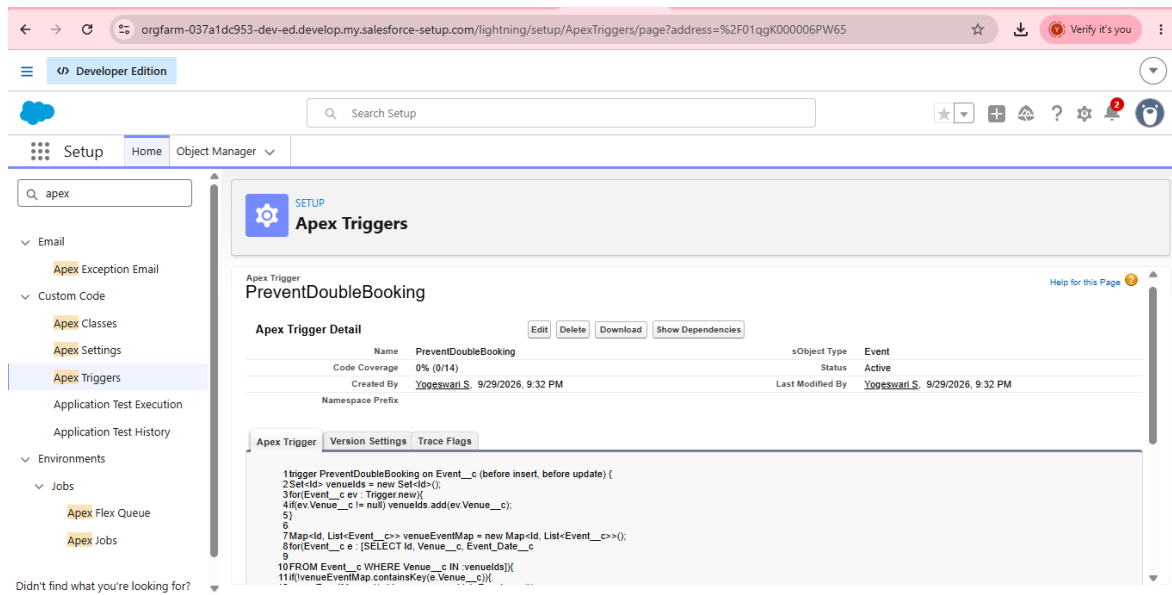


Figure 18: Apex trigger PreventDoubleBooking

## Nightly completion of past events

BatchCompleteEvents is a Batch Apex class. Its query selects every event whose Event Date is before today and whose status is not already Completed, and its execute method sets the status of each one to Completed. A short debug message is written when the job finishes.

ScheduleCompleteEvents is a Schedulable class that starts the batch with a scope of 200 records per batch, so the job continues to work as the number of events grows.

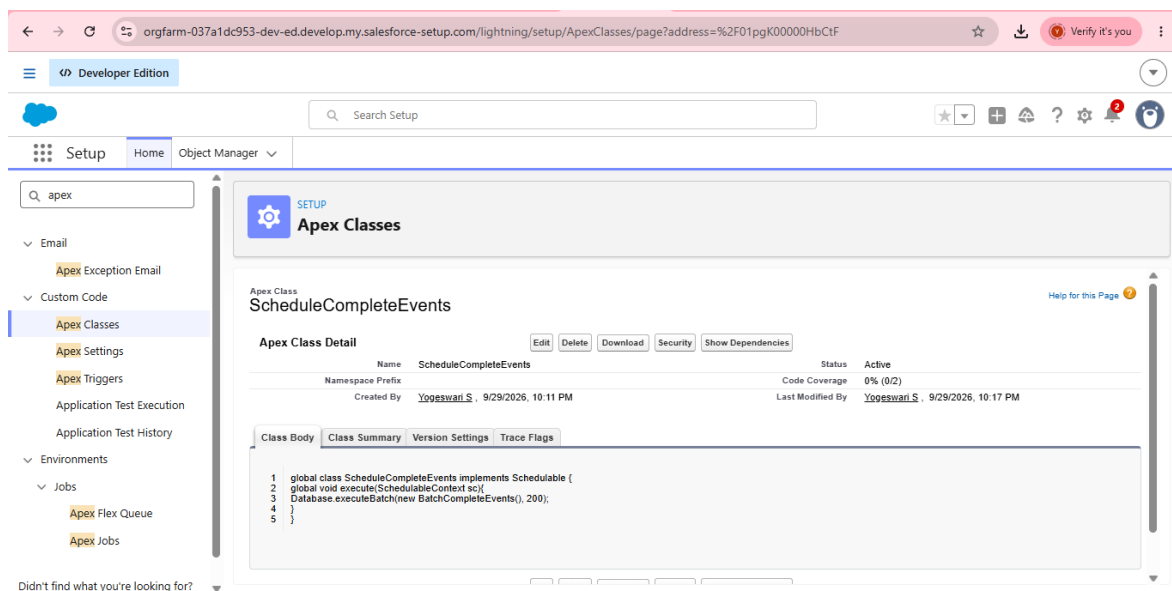


Figure 19: Schedulable class ScheduleCompleteEvents

The Apex Jobs page confirms that the schedule is working. ScheduleCompleteEvents was queued on 29 September 2026, and BatchCompleteEvents then completed with no failures on 30 September (1 batch processed), 1 October and 2 October 2026, each time at about 8:00 PM. The runs on 1 and 2 October show 0 batches because no event needed to be completed on those days.

orgfarm-037a1dc953-dev-ed.develop.my.salesforce-setup.com/lightning/setup/AsyncApexJobs/home

Developer Edition

Search Setup

Setup Home Object Manager

apex job

Environments Jobs Apex Jobs

Didn't find what you're looking for? Try using Global Search.

Apex Jobs

Click here to go to the new batch jobs page

Monitor the status of all Apex jobs, and optionally, abort jobs that are in progress.

Percent of Asynchronous Apex Used: 0%  
You have currently used 0 asynchronous Apex operations out of an allowed 24-hour organization limit of 250,000. To learn about how this limit is calculated and what contributes to it, see the Lightning Platform Apex Limits topic.

View: All Create New View

| Action | Submitted Date      | Job Type       | Status    | Status Detail | Total Batches | Batches Processed | Failures | Submitted By | Completion Date    | Apex Class             | Apex Method | Apex Job ID    |
|--------|---------------------|----------------|-----------|---------------|---------------|-------------------|----------|--------------|--------------------|------------------------|-------------|----------------|
|        | 10/2/2026, 8:01 PM  | Batch Apex     | Completed |               | 0             | 0                 | 0        | S_Yogeswari  | 10/2/2026, 8:01 PM | BatchCompleteEvents    |             | 707gK0000wXzJR |
|        | 10/1/2026, 8:00 PM  | Batch Apex     | Completed |               | 0             | 0                 | 0        | S_Yogeswari  | 10/1/2026, 8:00 PM | BatchCompleteEvents    |             | 707gK0000wMwbl |
|        | 9/30/2026, 8:01 PM  | Batch Apex     | Completed |               | 1             | 1                 | 0        | S_Yogeswari  | 9/30/2026, 8:01 PM | BatchCompleteEvents    |             | 707gK0000wBEF7 |
|        | 9/29/2026, 10:18 PM | Scheduled Apex | Queued    |               | 0             | 0                 | 0        | S_Yogeswari  |                    | ScheduleCompleteEvents |             | 707gK0000w0ecl |

Figure 20: Apex Jobs page showing the scheduled job and the completed batch runs

## 5.8 User Interface and Dashboard

The application is packaged as a Lightning App named Event planner. Its navigation bar has a tab for Clients, Events, Vendors, Venues and Feedbacks, plus Reports and Dashboards, so users can open records, create new ones and use list views without going through Setup. The New Event form shows how a coordinator enters an event: name, type, date, status, venue and client.

orgfarm-037a1dc953-dev-ed.develop.lightning.force.com/lightning/o/Event\_c/new?count=5&nooverride=1&useRecordTypeCheck=1&navigationLocat...

Developer Edition

Lightning Usage App

Events Recently Viewed

3 items • Updated a few seconds ago

1 Product Launch

2 Music Festival

3 Tech Conference

New Event

\* = Required Information

Information

Event Name Birthday party

Event Type Birthday

Event Date 10/17/2026

Event Status Completed

Venue Search Venues...

Owner Yogeswari S

Cancel Save & New Save

Figure 21: New Event form in Lightning Experience

For management we created the report “Upcoming Events by Month”, based on the Events report type. It groups events by Event Date month and currently shows 5 records: one in May 2026, one in August 2026, two in October 2026 and one in December 2026.

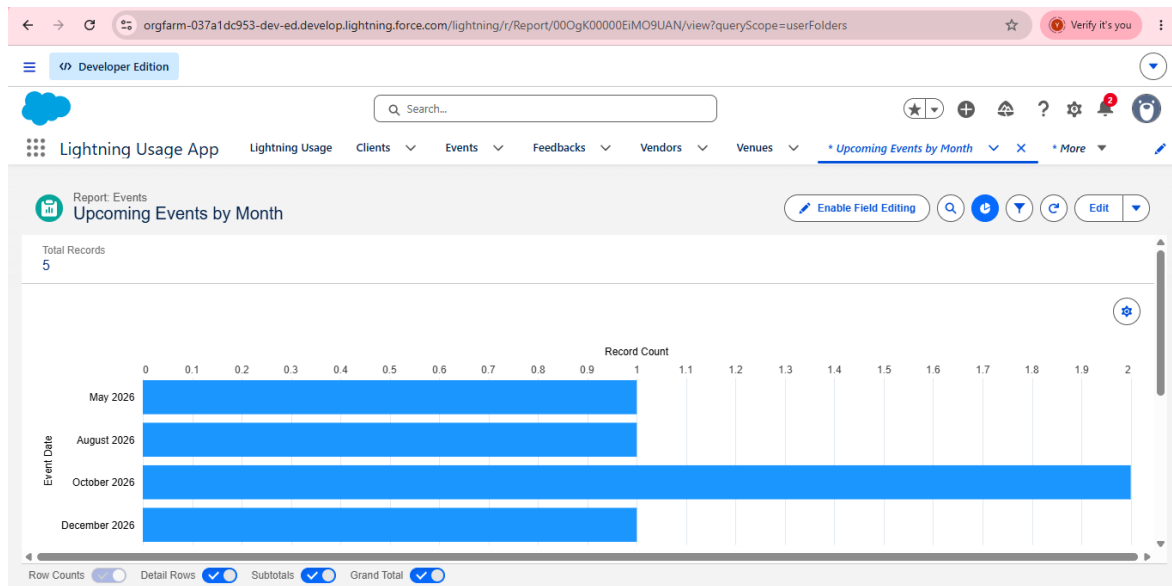


Figure 22: Report: Upcoming Events by Month

The EventForce Operations Dashboard displays this report as a funnel chart, with the total record count of 5 and a link back to the report. Opening the dashboard gives managers the monthly picture without having to run the report themselves.

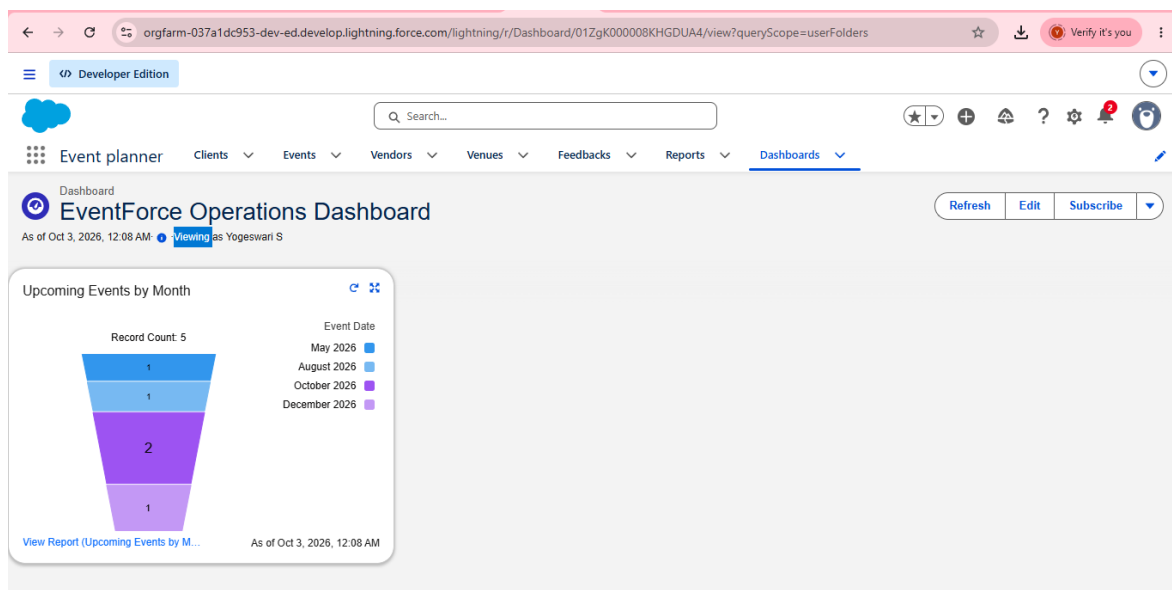


Figure 23: EventForce Operations Dashboard in the Event planner app

## 5.9 Security

Access to EventForce is controlled with roles, profiles, a permission set, organization-wide defaults and sharing rules so that each user sees only the records needed for their job. We created three roles for the application, Event Admin, Event Coordinator and Client, in the role hierarchy. The Event Coordinator role is also the recipient of the cancellation email alert.

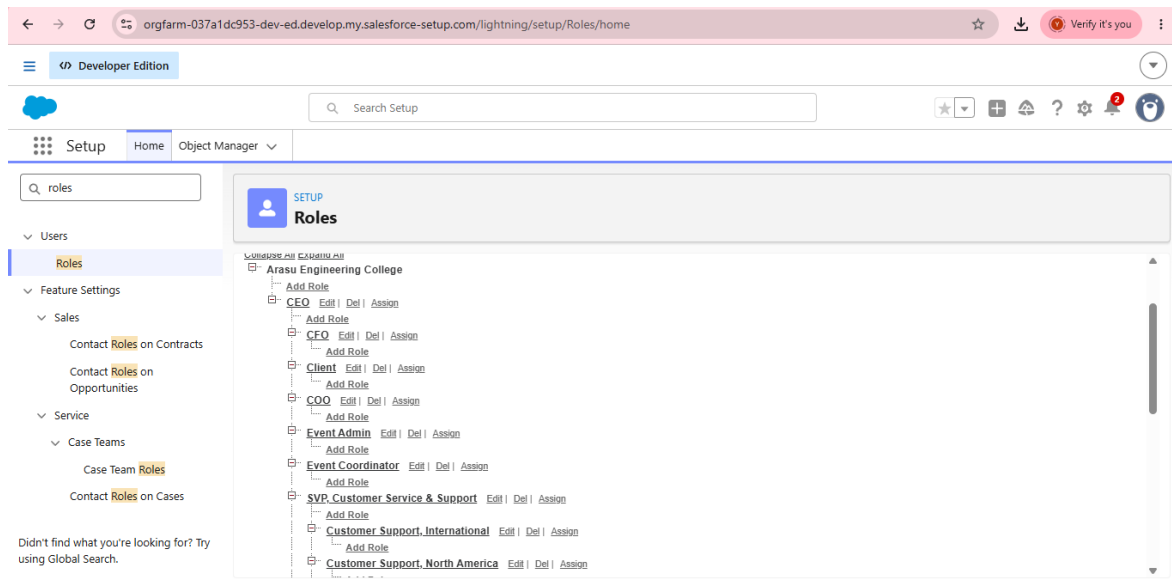


Figure 24: Role hierarchy with the Event Admin, Event Coordinator and Client roles

## 6. Testing

Each feature was tested from the Lightning interface. The table lists the test cases with the behaviour expected from the design.

| No. | Test case                                                | Expected result                                               | Evidence          |
|-----|----------------------------------------------------------|---------------------------------------------------------------|-------------------|
| 1   | Save a client with an invalid email such as abc@         | Save is blocked with “Please Enter Valid Email Address”       | [Add screenshot]  |
| 2   | Save a client using an email that already exists         | Save is blocked with a duplicate value error                  | Figure 25         |
| 3   | Create an event for a venue with no booking that day     | Event saves successfully                                      | Figure 21         |
| 4   | Create a second event at the same venue on the same date | Save is blocked: “This Venue is already booked on this date.” | [Add screenshot]  |
| 5   | Set an event to Confirmed, then to Canceled              | Venue becomes Reserved, then Available                        | [Add screenshot]  |
| 6   | Set Event Status to Pending Cancellation                 | Approval request created; coordinators alerted                | [Add screenshot]  |
| 7   | Event is three days away                                 | Client receives the reminder email                            | [Add screenshot]  |
| 8   | Nightly job runs                                         | Past events become Completed; no failures                     | Figure 20         |
| 9   | Open the report and the dashboard                        | Upcoming events are shown by month (5 records)                | Figures 22 and 23 |

## 7. Challenges Faced and Fixes

Two problems had to be solved before the application worked as designed.

## 7.1 Apex compile errors

Some of our Apex classes and triggers did not save at first because the Setup editor reported compile errors. We read the line number and message shown with each error, corrected the code and saved again until every component showed the status Active. The components depend on each other (EventTrigger13 calls VenueStatusHelper, and ScheduleCompleteEvents starts BatchCompleteEvents), so the called class must exist and compile before the code that uses it; we therefore saved the classes first and the triggers and scheduler afterwards. Once all errors were cleared, the scheduled batch ran without failures (Figure 20).

## 7.2 Duplicate value error while creating clients

After the code was working, creating a new client failed with the message “duplicate value found: Email\_\_c duplicates value on record with id ...”. The Email field on the Client object is unique, so Salesforce refuses a second client with an email address that already belongs to another record. We fixed it by giving each client their own email address. If shared addresses are ever needed, the unique setting on the field can be relaxed.

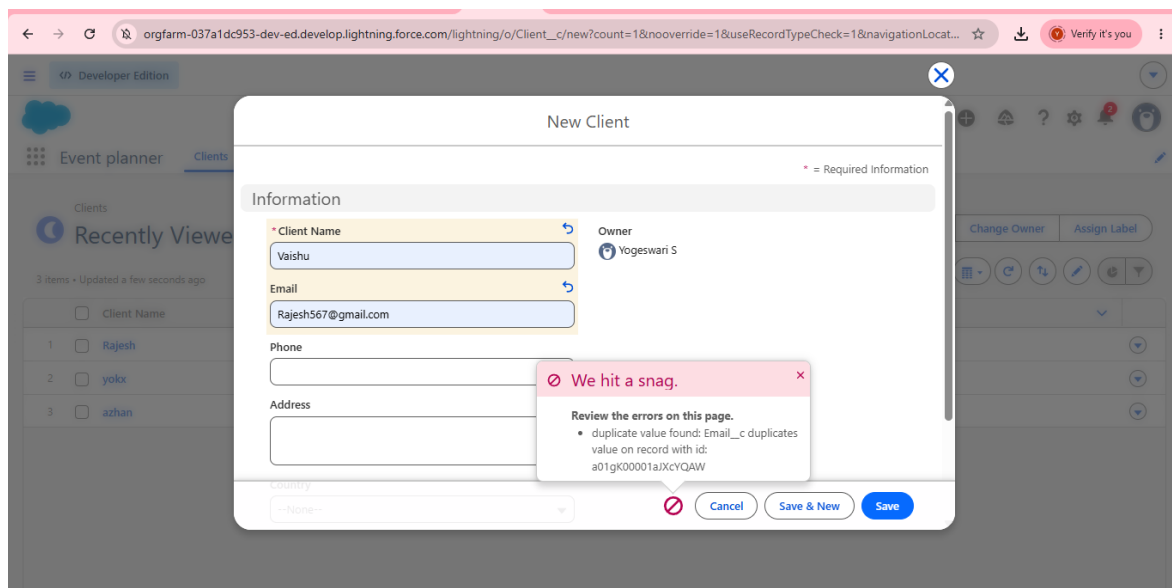


Figure 25: Error shown when a client is saved with an email that already exists

## 8. Limitations and Future Scope

The Apex triggers and classes currently show 0% code coverage in Setup because test classes have not been written yet. This does not stop them working in a Developer Edition org, but Salesforce requires at least 75% coverage before code can be deployed to production, so writing test classes is the first improvement planned.

Other ideas for future work:

- Online payment integration so clients can pay deposits from Salesforce.
- A mobile app or Experience Cloud portal for clients to track their events.
- More reports and dashboard charts, such as events per venue and feedback ratings.
- Automatic emails for approval results and for requesting feedback after an event.

## 9. Conclusion

EventForce Management System brings the core work of an event planning company into one Salesforce application. The data model keeps clients, events, venues, vendors and feedback connected; the validation rule and formula field protect data quality; the approval process controls cancellations; and the Flow and Apex code take care of reminders, venue availability, double-booking prevention and nightly completion without manual effort. Building it gave our team practical experience with Salesforce objects, relationships, declarative automation, Apex, scheduled jobs and dashboards, and the same design can be extended with payments, a client portal and richer reporting.

## Appendix A: Apex Source Code

### EventTrigger13

```
trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(trigger.new);
}
```

### ScheduleCompleteEvents

```
global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc) {
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}
```

### VenueStatusHelper

```
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events) {
        List<Venue__c> venues = new List<Venue__c>();
        for (Event__c ev : events) {
            if (ev.Venue__c != null) {
                if (ev.Event_Status__c == 'Confirmed') {
                    venues.add(new Venue__c(Id = ev.Venue__c,
Availability_Status__c = 'Reserved'));
                } else if (ev.Event_Status__c == 'Canceled') {
                    venues.add(new Venue__c(Id = ev.Venue__c,
Availability_Status__c = 'Available'));
                }
            }
        }
        if (!venues.isEmpty()) update venues;
    }
}
```

### PreventDoubleBooking

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for (Event__c ev : trigger.new) {
        if (ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }

    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for (Event__c e : [SELECT Id, Venue__c, Event_Date__c
                        FROM Event__c WHERE Venue__c IN :venueIds]) {
        if (!venueEventMap.containsKey(e.Venue__c)) {
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }

    for (Event__c ev : trigger.new) {
        if (ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)) {
            for (Event__c existing : venueEventMap.get(ev.Venue__c)) {
                if (existing.Event_Date__c == ev.Event_Date__c && existing.Id !=
ev.Id) {
                    ev.addError('This Venue is already booked on this date.');
```

```

    }
    }
}

```

## BatchCompleteEvents

```

global class BatchCompleteEvents implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Event_Date__c, Event_Status__c
            FROM Event__c
            WHERE Event_Date__c < TODAY
            AND Event_Status__c != 'Completed'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<sObject> scope) {
        List<Event__c> events = (List<Event__c>) scope;
        for (Event__c ev : events) {
            ev.Event_Status__c = 'Completed';
        }
        update events;
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Past events marked as Completed');
    }
}

```